

Zombie AI Behaviour System in a 2D Platformer Game

Anshul Shashikant Keng

Student ID: 25028367

University of Limerick

AI for Games

Word Count: ~1700

Video:-

https://drive.google.com/file/d/1kp0A6bNFYw_eRBfcEYiT2-QDk5ufrd5/view?usp=drive_link

Code :-

<https://drive.google.com/file/d/1JUzj0rN18t6TPB7tUP1U91lo3z3ULMCA/view?usp=sharing>

Table Of Contents

1. Introduction

2. Analysis

3. Reflection

4. Conclusion

1. Introduction

This report examines the development and behaviour of the **Zombie AI system** implemented inside a custom built 2D platformer game. The artefact contains three major gameplay elements a player, an enemy (zombie), and an bat companion. Among these, the zombie agent represents the central challenge of the game, driving tension, combat and survival. The zombie actively seeks the player's location, moves toward them, attacks at close range, deals damage, and can be defeated through repeated hits. When killed, the zombie drops a health pickup which helps the player recover and continue progressing. This creates a reward-based risk loop common in modern action platformers.

My task in this project was to design, implement and refine the complete **Zombie AI system**. This included enemy behaviour, pathing, attack logic, hit detection, player damage response, death event handling and health drop mechanics. The zombie had to feel believable not too easy, not impossibly difficult and most importantly dynamic. A zombie that only stands still is harmless, while a zombie that instantly kills the player ruins the game. The goal was to find a behavioural balance where the enemy appears threatening but beatable, predictable in behaviour but still demanding enough attention to engage the player.

The AI solution is based on a combination of distance checking, movement steering, animation triggered attack states, and controlled cooldown timing. This approach sits between simple behaviour trees and advanced pathfinding; it is lightweight, reactive and easy to scale into waves or variations. Unlike scripted animation only enemies, this zombie behaves based on environment and player position. Unlike complex navigation systems, this AI trades path planning for direct pursuit, which suits the linear level design of a side scrolling game.

The purpose of this report is to analyse the logic behind the Zombie AI, explain how each behaviour was implemented, and evaluate the outcomes, challenges and future development opportunities. While the final game contains additional systems, including coin spawning and a optional companion bat, this document focuses strictly on the **Enemy (Zombie) AI** as the primary deliverable.

2. Analysis :

2.1 Why this AI method was chosen

For this project, I chose **distance-based state switching** as the core AI technique. The zombie constantly measures its distance from the player. If the player is far away, the zombie will walk toward them. If close, it will stop, face the player and attack. This creates a natural predator chase effect without requiring high-cost grid navigation, raycasting or NavMesh navigation. In a 2D environment where paths are mostly straight and open, this method is efficient and effective.

This method also supports wave gameplay. By adjusting health and attack values, new zombies can scale in difficulty over time. Their AI behaviour does not need rewriting; increasing numbers or stats simply increases difficulty. This mirrors real games like Dead Cells, Terraria, and Hollow Knight, where enemies share behaviour frameworks but vary in strength and aggression.

The zombie AI also uses an **attack cooldown mechanic**. Without cooldown, the player would lose full health instantly on contact. With cooldown, the zombie attacks rhythmically, giving the player time to escape or fight back. The player's attack is also damage based rather than instant kill. The zombie requires multiple hits to defeat, and each fight becomes a small challenge.

Finally, zombies drop health pickups upon death. This ties combat to progression and risk-reward design. The player only recovers HP if they are brave enough to fight instead of running. This decision-making loop is behaviour reinforcement good AI makes players think, not only react.

2.2 ZombieHealth Script Behaviour

The ZombieHealth script manages enemy durability and its destruction. It stores `maxHealth`, tracks `currentHealth`, receives damage, and dies when the health reaches zero. The death event then spawns a health pickup, rewards the player, and removes the zombie from the scene.

Component Role

- Tracks zombie

- health
 - Accepts damage from player attacks
- Detects death condition
- Triggers zombie removal
- Spawns a healing pickup as loot

This script makes enemy combat meaningful. The zombie is not decorative it resists, survives multiple hits, and eventually falls. The player experiences progress and satisfaction through damage feedback, especially when combined with hit sound, animation and logs. The health drop system also shapes gameplay pacing. After difficult battles, the player heals and continues survival is earned, not given.

2.3 ZombieAI Script Behaviour

The ZombieAI script is the brain of the enemy. It detects the player's position, moves toward them, stops when close enough, and begins attacking. The logic can be broken down into 3 core behaviours:

1. Player Tracking

The AI finds the player using their tag. Every frame, it measures distance using `Vector2.Distance`. If distance is greater than stop range, the zombie moves horizontally toward the player. This creates the chase behaviour.

2. Attack Execution

When the player enters attack range and the cooldown timer has reset, the zombie deals damage to the player's health script. Attacks cannot occur continuously delay creates fairness and tension.

3. Death Integration

When health reaches zero, `ZombieHealth` triggers `Die()`, removing AI control and spawning rewards. This closes the enemy loop encounter, combat, reward, continuation. These behaviours combined create a believable enemy archetype something moving, hunting,

dangerous and defeatable. No pathfinding complexity was required because level layout supports direct motion. This ensures CPU-low AI that can scale to multiple zombies without performance loss.

3. Reflection :

3.1 Outcome of Behaviours

The Zombie AI achieves intended gameplay goals. It tracks the player reliably, responds to distance, and attacks consistently while respecting cooldown. Players lose health when hit and must strike repeatedly to kill the zombie, requiring measured combat instead of button spam. Health drops reinforce combat encounters meaningfully every fight is a risk that may reward survival.

Combat feels dynamic: if the player runs, the zombie follows. If the player fights, the zombie retaliates. If the player wins, the zombie dies and drops healing. The outcome is a loop of challenge and reward, suitable for a continuous scrolling 2D action platformer.

3.2 Issues & Limitations Faced

Several technical challenges occurred during development:

- Attack logic originally triggered too frequently causing instant death
- Early zombies were harder than late-wave zombies when upgrades reversed incorrectly
- Animation integration temporarily broke death execution
- Zombies occasionally ignored collision layers when physics setup was incorrect

These were solved through cooldown implementation, health balancing per wave, removal of animation constraints, and careful physics collider configuration.

3.3 Limitations of Current AI

Despite successful behaviour, current zombies still lack:

- Strategic movement :they only move in straight pursuit
- Avoidance or positioning behavior : they overlap when multiple spawn
- Awareness beyond proximity : no sight, hearing or pathfinding

These elements could be enhanced later.

3.4 Future Development Directions

There are multiple directions to evolve this enemy AI beyond its current system:

- Introduce raycast vision
- Add pathfinding for complex terrain
- Improve group behaviour coordination
- Create tiered variants (fast zombies, tank zombies, ranged zombies)
- Add dodge or retreat logic for unpredictability

These additions would transform zombies into more tactical and intelligent adversaries, scaling the game into deeper combat and survival challenges.

4. Conclusion

The Zombie AI developed for this project demonstrates successful implementation of enemy logic suitable for a 2D platformer. Through distance based movement, attack cooldown systems, health tracking and reward drops, the AI produces engaging combat rather than static scripted behaviour. The player must engage strategically, balancing attack and retreat while zombies pressure forward. The system is expandable, lightweight and fits performance requirements well.

This work strengthened my design understanding of behaviour-driven AI, damage systems, combat balance, and reward loops. The AI behaves logically, consistently, and enhances gameplay through tension and challenge fulfilling the expectations of this assignment effectively.

